

loci

Scoped memory for coding agents

ENGINEERING & ARCHITECTURE DOCUMENT

Version 1.0 · 8 September 2026

Repository state: `main @ a8a4673`, release 0.5.0

All figures independently verified against the working tree

8,105

LINES OF SOURCE

279

AUTOMATED TESTS

29

MODULES

85COMMITTS SINCE 27 AUG
2026

Contents

1. Overview

- 1.1 The two claims
- 1.2 What is not here

2. At a glance

- 2.1 Verified metrics
- 2.2 Composition
- 2.3 The naming split

3. The problem, measured

4. Architecture: the query path

5. The routing decision

- 5.1 The scoring
- 5.2 The three refusal gates
- 5.3 Frame nouns

6. Scopes and groups

- 6.1 Mode does two different jobs
- 6.2 Two files, on purpose
- 6.3 Splitting is off by default, and the reason is measured

7. The two stores

- 7.1 Fusion, and what it costs
- 7.2 Reranking is off by default, and says why

8. Abstention

- 8.1 An abstention carries its shortlist

- 8.2 Two things that look like confidence and are not

9. Measurement

- 9.1 The evaluation runs on your corpus, not a benchmark
- 9.2 The measured run
- 9.3 Calibration

10. Cross-project edges

11. Redaction

12. Performance

13. From inside an AI client

14. Design decisions and their reasons

15. Open problems

16. Conclusion

Appendix A: Verification log

- A.1 Tests
- A.2 Scale
- A.3 Routing evaluation
- A.4 Calibration
- A.5 Cross-project edges
- A.6 Routing constants

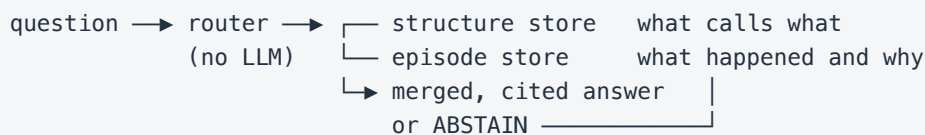
Appendix B: What it writes

1. Overview

loci is a memory system for coding agents. It is a router in front of two stores, and the whole design rests on one decision: it works out which project a question is about at **read** time, so you never have to name a namespace at **write** time.

An agent working across many repositories has two kinds of question and neither is answered well by the tools available. *What calls `run\_agent\_turn`?* is a structure question, and knowledge graphs answer it. *Why was the admin session cookie dropped on localhost?* is a history question, and verbatim-recall systems answer it. Neither system answers the other's question, and both of them make you say which project you mean when you write the memory down.

That last part is the real problem. At write time, an agent finishing a piece of work does not reliably know which namespace the conclusion belongs to, and a namespace chosen wrongly is a memory that can never be found again. At read time the question itself carries the evidence: its vocabulary, the directory you are standing in, and the project names it mentions. loci moves the decision to where the evidence is.



One question path serves *"how does auth work here"* and *"have I solved this in any project"*. Only the size of the scope set changes.

1.1 The two claims

Merging projects into one index destroys retrieval, and the damage is invisible. This is measured rather than argued, and Section 3 gives the numbers. The failure is not that a merged index is worse on average. It is that a merged index looks excellent whenever the answer happens to live in the largest project and collapses when it does not, which is the same failure in both directions and visible in only one.

Abstention is a feature, at both layers. A retrieval system that always returns its best match will confidently answer questions it has no material for, because "best" is a ranking and a ranking always has a top. **ABSTAIN** in loci means *ask the user*, not *pick the biggest*, and both the router and the stores can refuse independently.

Neither claim is asserted. Every constant in the routing function was fitted or measured, several were tried and rejected with the measurement recorded, and Section 14 collects the ones where the obvious implementation is wrong.

1.2 What is not here

No model in the query path. Routing is a dictionary lookup per query token, deterministic and sub-millisecond. Embeddings, when enabled, run locally from a small model you opt into. Nothing leaves the machine.

The base install pulls four small packages and downloads no model. That is a design position rather than an accident: a memory tool whose pitch is that it holds your private working history should not require sending that history anywhere, and should not require a vector database to be useful.

2. At a glance

2.1 Verified metrics

MEASURE	VALUE
Source	8,105 lines across 29 Python modules
Test code	5,446 lines in one pytest file
Tests	279 passed, 0 failed, 0 skipped in 26.5s
Commit history	85 commits, 27 August to 6 September 2026
Releases	5, from 0.1.0 on 28 August to 0.5.0 on 6 September
Base runtime dependencies	4
Optional extras	4 (<code>graphify</code> , <code>embeddings</code> , <code>rerank</code> , <code>mcp</code>)
CLI commands	20
MCP tools exposed	3
Routing latency	sub-millisecond, flat from 25 to 100 scopes
Development corpus	15 registered scopes

2.2 Composition

MODULE	LINES	JOB
<code>cli.py</code>	952	The command surface, 20 commands
<code>router.py</code>	885	The routing decision, and every constant behind it
<code>backends/episodes.py</code>	618	The episode store: chunking, fusion, the two-tier gate
<code>scopes.py</code>	493	The scope registry, scanning, aliases
<code>edges.py</code>	493	Cross-project references, declared and spawned
<code>ask.py</code>	440	The query path above the stores
<code>index.py</code>	373	Index build, fingerprinting, reuse
<code>setup.py</code>	348	The one-pass first run
<code>eval.py</code>	339	Auto-labelled routing evaluation
<code>calibrate.py</code>	290	Fitting the evidence floor to a corpus
<code>identity.py</code>	286	Signboards: what a project is called from outside

MODULE	LINES	JOB
<code>update.py</code>	276	The refresh chain
<code>backends/docstrings.py</code>	227	Docstring and comment-block collection
<code>provenance.py</code>	212	Ownership read from git
<code>doctor.py</code>	208	Coverage gaps, and the command that fixes each
<code>mcp_server.py</code>	207	The stdio MCP server
<code>backends/graphify.py</code>	206	The structure-store adapter
<code>redact.py</code>	89	Credential redaction at collection time

The distribution of that table is the argument. `router.py` is 885 lines to make one decision, and `redact.py` is 89 lines to make a decision that has no interesting cases at all. The size difference is the point: the routing decision is where every measured finding in this document landed, and the redaction decision is one where the correct answer was known in advance and the only engineering was making it unavoidable.

2.3 The naming split

The distribution is `'loci-mem'`. The command, the import and the project are all `loci`. PyPI's `loci` is an unrelated outlier-detection package abandoned in 2018, so the split is forced, and it is the same shape as `python-dateutil` installing as `dateutil`.

That detail earns a section because it turned out to be load-bearing in a way nobody planned. A project has several names, and Section 10 exists because an edge in the corpus mentions whichever one it happens to use. `loci` is the case that forces the shape: `loci-mem` on PyPI, `loci` to import, `loci` to run, `3M1RY33T/loci` on GitHub. Four names, one project, and the only one any dependency declaration actually mentions is the third.

3. The problem, measured

Put every project in one index and the largest one wins regardless of the question. Measured against a merged graph of ten real repositories:

QUESTION	ON-TOPIC NODES RETURNED
"why was the admin session cookie dropped on localhost?"	18% , 19 of 31 came from the biggest project
"what happens when a user clicks Save as ZIM?"	2% , 61 of 62 came from the biggest project
"how does the reserved landing step work?"	98%, and the biggest project <i>was</i> the answer

That third row is the trap. Merged retrieval looks excellent whenever the answer happens to live in the largest corpus and collapses when it does not. It is the same failure in both directions and visible in only one, which means a merged index evaluated on a handful of questions can pass while being badly broken, and the questions it passes on are exactly the ones a developer would think to try first.

The mechanism is size, not quality. Section 14.6 covers the normalisation that follows from it, and the finding worth stating here is that with one scope 67 times larger than another, **ordinary English words appear in exactly one scope and score as maximally discriminative for it**. A large project does not out-compete a small one on relevance. It out-competes it on vocabulary coverage, which is not the same thing and is not what anyone intends to rank on.

4. Architecture: the query path

The diagram in Section 1 is the elevator version. In full:

```

loci ask "why was the session cookie dropped on localhost?"
|
|  --scope NAME jumps past routing to the fan-out, and drops the episode
|  gate with it: that gate exists to stop an answer arriving from the
|  wrong scope, and you have just named the right one.
|
▼
confinement                                     groups.py
|
|  reads the registry and groups.json. Never the index, never a model.
|  --group X → eligible = members of X, in ALL THREE modes
|  else cwd → the strictest group of the scope you are standing in
|
▼
route                                           router.py
|
|  reads scope_index.json: token → {scope: node_df}. One dict lookup per
|  query token. Deterministic, sub-millisecond, no model call.
|
| → ABSTAIN, naming the cause, listing the scopes with a claim on the
|    question and what each one holds, naming the flag that
|    fixes it, and querying nothing. An outcome, not an error.
|
▼
selected scopes, at most 3      one thread each; neither store is ever
|                               queried across a scope boundary
| — scope — scope — scope
|
|  expand the question against THIS scope's postings, a dict lookup,
|  so a token the scope does not have cannot be invented
|
|  ┌ structure store  graphify query, what calls what, with file:line
|  └ episode store   BM25 + char 3-5 gram + embeddings, fused, gated
|
▼
merged, cited answer, one block per scope

```

Two properties of that shape are worth naming because they are unusual.

Both stages refuse independently. The router can decline to select a scope at all, and a scope it did select can still hand back nothing. Those are different failures with different fixes, and collapsing them into one "no results" would hide which happened.

Nothing is queried across a scope boundary. Query expansion is done against each scope's own postings, so a token that scope does not hold cannot enter its query. That is what makes the scope isolation real rather than a filter applied afterwards.

5. The routing decision

Everything above the fan-out is one function. It scores every scope in the corpus, then asks three refusal questions in a fixed order.

5.1 The scoring

```
evidence base, for EVERY scope:
    sum(scope-idf × prominence) / query tokens / size^0.15

then, in this order:
    demoted? base × 0.5      applied BEFORE the boosts, so it cannot invert one
+ 6.00      the question names the project outright
+ 4.00      cwd is inside the project tree, deepest containing scope only
+ 0.15      recency, a tiebreak and no more
```

The ordering of the penalty against the boosts is the kind of detail that only matters once. A group penalty applied *after* a 6.0 alias boost is a rounding error; applied before, it does what it was written to do.

Weight is not usefulness. The working directory carries a boost of 4.0 against an alias's 6.0, and it is the signal that carries most real questions, because most real questions name no project at all. "*How is this deployed?*" and "*how do I run the tests?*" route correctly **100% of the time with cwd** and are unanswerable without it. cwd is the primary signal here, not a tiebreaker.

5.2 The three refusal gates

GATE	FIRES WHEN	MEANING
<code>out_of_group</code>	the corpus-wide winner is outside a hard group <i>and</i> would itself have routed	the best answer is somewhere you said not to look
<code>deictic</code>	the question points at a subject it never names	no vocabulary can identify the subject
<code>no_evidence</code>	none of three OR'd evidence tests passes	the question's words are not in the corpus

Four things in that structure are load-bearing and none of them are obvious.

`forced` is an escape, not a signal. An alias hit or a cwd hit contributes nothing to either evidence count, so a scope winning purely on 6.0 or 4.0 reads as zero evidence to the gates below it. It skips them instead. Deixis is only a problem when nothing else has identified the subject, so a question that says "it" *and* names a project is unaffected.

`out\_of\_group` is judged against the whole corpus, not the group. It fires only when the corpus-wide winner is outside the group *and* would itself have routed. Without that second test it swallowed the other

two reasons entirely: on a question matching no vocabulary, every scope scores near zero and the winner is whoever took the 0.15 recency tiebreak.

The three evidence gates are OR'd because they fail on different question shapes. A short question about a rare symbol has high evidence and a low matched-token count. A long question about a familiar subsystem has the reverse. Requiring all three abstains on both.

Enumeration inverts every other gate. Every gate above asks *is there enough evidence for one scope*, and a question about something several projects share splits its evidence across them by construction, so the more projects genuinely share a term, the less likely all of them come back. Measured, "*which of my projects use Cloudflare workers or D1?*" returned one owner of two: 2.098 against 1.0867, where the 0.85 widening ratio needed 1.783. Set mode drops the ratio entirely and asks each scope the same routability question in turn against a floor discounted to 0.8, because a member of a set legitimately carries less evidence than a lone answer.

5.3 Frame nouns

Enumeration is detected grammatically, by a closed class of markers, with no threshold. The frame nouns are then stripped from the query, which is the deixis insight one level down: `projects` is scaffolding, not vocabulary.

It is not a cosmetic step. In the development corpus `projects` is held by exactly two scopes, so it scored as discriminative evidence *for those two* and dragged both into every "which of my projects" answer. Stripping it moved negative-family abstention from 66.7% to 88.9%.

6. Scopes and groups

Groups are overlapping labels over a flat scope set. There is no tree anywhere and nothing is ever merged.

scope		the groups it carries		
delroy	monorepo	me	delroy	
delroy/glasses	sub-scope	me	delroy	
delroy/client	sub-scope	me	delroy	client:acme
acme-api	repo	me		client:acme
vendorlib	repo			vendor:someorg

`delroy/glasses` is not stored *inside* `delroy`. It carries the label `delroy` exactly as it carries `me`. `client:acme` spans a monorepo sub-project and an unrelated repository, which a hierarchy could not express at all.

6.1 Mode does two different jobs

What a group does to a question is its **mode**, and the mode answers two different questions depending on how the group was reached.

	REACHED THROUGH CWD	NAMED WITH <code>--GROUP X</code>
<code>explicit</code>	nothing	confines to X's members
<code>soft</code> (default)	every outside scope's evidence base $\times 0.5$	confines to X's members
<code>hard</code>	confines, and abstains when the best answer is outside	confines, and abstains when the best answer is outside

The right-hand column is the counter-intuitive half, so it is measured rather than asserted. On a three-scope fixture with `alpha` and `beta` in group `team` and `vend` outside it, one question under four policies:

```
loci route "how does the gizmo parser emitter handle sprocket calibration"

unconfined                -> vend, alpha
--group team explicit      -> alpha
--group team soft          -> alpha
--group team hard          -> ABSTAIN (out_of_group)
                           candidates: alpha (sprocket), beta (calibration)
```

All three modes dropped `vend`, the scope that won unconfined. `--group` is the user asserting the answer is in here; the mode decides only what happens when it is not.

6.2 Two files, on purpose

Membership lives in the scope registry and mode lives in `groups.json`, and they are separate files for a reason that is easy to get wrong. `scopes.json` is machine-managed and rewritten wholesale by every scan. `groups.json` is the user's and must survive one. Putting policy in a file that a re-scan overwrites is a bug that only appears the second time someone runs `loci scan`.

6.3 Splitting is off by default, and the reason is measured

A monorepo can become one scope per package. `--split` takes anything carrying `package.json`, `pyproject.toml`, `Cargo.toml` or `go.mod` one level down, and a repo-local `.loci.json` names the rest.

`--split` is off by default, and the reason is measured rather than cautious. A new scope's aliases include its bare directory name, and an alias outranks the working directory 6.0 to 4.0. On the development corpus, splitting a repository holding `glasses/` sent eight hand-written questions about a *different* project to `Delroy/glasses`. Seven of them had routed correctly before the split, and six of the eight reverted when the alias boost was zeroed. A Jekyll `_site/` build directory became a scope of its own on the same run.

A `.loci.json` written by hand does not have that problem, and is therefore honoured with or without the flag: those sub-projects were named deliberately, so the person who named them knows what they are called.

7. The two stores

Structure store. What calls what: symbols, files, references, traversals with `file:line` citations. Supplied by graphify through an adapter.

The adapter is the design decision. graphify is a dependency, never a fork: loci reads its `graph.json` data contract through `backends/graphify.py`, so a different extractor can be substituted without touching anything above `backends/`. That is 206 lines buying the ability to be wrong about the extractor choice.

Episode store. What happened and why: README and docs, git commit bodies, docstrings and comment blocks, and any notes pointed at it. Stored verbatim, chunked on heading boundaries, redacted before it is written.

7.1 Fusion, and what it costs

Episode search fuses three rankers, renormalising over whichever produced a result:

RANKER	CATCHES
BM25	exact terminology, rare identifiers
char 3-5 grams	morphology and casing, <code>samesite</code> against <code>SameSite=None</code>
embeddings (bge-small, local, optional)	meaning without shared words

The character-gram ranker is the one that is easy to leave out and should not be. It recovers a class of match that neither of the others sees, and it costs no model.

Adding embeddings destroyed abstention. Every chunk gets a nonzero score, so an absolute floor stops firing and the store answers everything. The fix is a **two-tier gate**: a hit must be lexically grounded or semantically confident. A lexical gate alone would reject exactly what embeddings were added for, which is why the two conditions are OR'd rather than AND'd.

7.2 Reranking is off by default, and says why

Cross-encoder reranking is available with `--rerank` and is off by default. Measured: **2 of 6 to 3 of 6 precision at 1, at roughly 96ms per query, with two small regressions**, on six cases whose labels are themselves arguable.

That is a weak result honestly reported, and the shipping decision follows from it rather than from taste. A feature that improves one case, regresses two, costs 96ms and is evaluated on six arguable labels is a feature that should be available and off.

8. Abstention

8.1 An abstention carries its shortlist

An abstention that only says *no* leaves the asking to the user. loci's carries the shortlist it refused to choose between, and the terms that put each candidate there:

```
loci ask "how is caching handled?"

ABSTAINED - not enough of the question exists in any project.
  candidates: Delroy (handled, caching), odysseus (handled, caching),
              tensor-serve (caching)
  re-run with --scope <name>, or from inside the project directory.
```

A scope earns its place on that list by holding a term **at most half the corpus holds**. Holding one is not enough: `change`, `handled` and `work` sit in nearly every project, so matching them is a coincidence rather than a claim. Measured on the development corpus, that share is the difference between naming three projects and listing all fourteen in score order, which is the registry rather than a shortlist.

When nothing clears that bar there is no shortlist to print, and saying so is the answer:

```
loci ask "xyzzz plugh frotz quuz"

ABSTAINED - not enough of the question exists in any project.
  no project holds a distinctive term from this question
  -- `loci doctor` shows what is not indexed.
```

8.2 Two things that look like confidence and are not

Both were implemented and both were removed, with the measurement recorded.

A margin or ratio test does not measure confidence. Both invert on real data. "*How do I fix this bug?*", which should abstain, produces a routing margin of 0.94, while a correctly-routed question produces 0.18. Nonsense scored 0.608 against a large scope with a **higher** top-to-p90 ratio than a genuine question against a small one.

A matched-token count is not evidence. Compared against auto-labelled questions, the count does not separate routable from unroutable at all. It inverts. Summed, size-normalised token evidence does. The count survives only as one of three OR'd gates, where its job is to catch the long-question case the evidence sum handles badly.

What actually separates them is whether the question's words exist in the corpus at all. That is a different question from *which scope scores highest*, and every failed attempt above was an attempt to derive the first from the second.

9. Measurement

9.1 The evaluation runs on your corpus, not a benchmark

Every accuracy figure in this project was fitted and measured against one person's repositories. That is a claim a reader would otherwise have to take on trust, so `loci eval` ships and runs against whatever is registered, in a few seconds, with no labelling.

It only asks questions whose correct answer is known by construction:

FAMILY	HOW GOLD IS KNOWN
deictic + cwd	asked from inside a scope, so the answer is that scope
deictic, no cwd	nothing says which project, so abstaining is the only correct answer
unanswerable	nonsense no software corpus can answer
signature	built from each scope's own most distinctive vocabulary
detailed	longer questions in ordinary vocabulary
contended	a term two scopes share; both should come back

9.2 The measured run

Run against the 15-scope development corpus during preparation of this document:

15 scopes | random guessing would score 6.7%

family	n	correct	scopes	what it measures
deictic + cwd	60	100.0%	–	should route to the scope you are in
deictic, no cwd	4	100.0%	–	should abstain
unanswerable	3	100.0%	–	should abstain
signature	15	93.3%	1.3	are your scopes distinguishable?
detailed	30	86.7%	–	longer questions, ordinary vocabulary
contended	12	75.0%	1.7	two scopes share the term; both back

Three of those numbers deserve comment rather than celebration.

The signature family is an upper bound and it is no longer perfect. It asks each scope about its own rarest words, so a real question phrased in shared vocabulary does worse by construction. It scored 100% on a ten-scope corpus when the constants were fitted and scores 93.3% on fifteen. A score falling as the corpus grows is the expected direction, and the honest reading is that the fitted constants were fitted on a smaller and easier corpus than the one they now run against.

`contended` at 75% is the weakest number here and it is the most informative one. It is the family that tests the case the whole tool exists for: two projects legitimately share a term and both should come

back. Three of twelve did not.

The deictic families are at 100% and that is the design working. Sixty questions that name no project route correctly on the working directory alone, and seven that could not be routed by anything abstain rather than guess.

9.3 Calibration

The evidence floor is the one constant that depends on the corpus, because it depends on how much vocabulary the projects share. `loci calibrate` fits it from the same auto-labelled questions:

evidence floor	5.411
semantic floor	0.639 (mean per-scope, 8 scopes fitted, bands separate by +0.114)
fitted from	155 routable + 7 unroutable questions, none hand-labelled
routable band	from 2.507
unroutable band	up to 5.092
classifies	93.8% of its own samples correctly

Evidence alone separates routable from unroutable questions on this corpus.

The shipped default is 7.6. This corpus fits 5.411, which is a 29% difference on the single constant that decides whether the tool answers at all, and it is the reason `calibrate` exists as a command rather than as a comment in the source.

When the bands overlap, the command says so. **An overlap warning is information about the projects, not a defect:** they share enough vocabulary that evidence alone cannot always tell them apart, and `cwd` or `--scope` should carry more of the weight. On this run they separate, by +0.114.

10. Cross-project edges

Release 0.5.0 exists because of one question the router could not be asked.

"In which project am I using another one of my projects?" Asked verbatim in three sessions on 6 September 2026, all three abandoned:

```
ABSTAINED – not enough of the question exists in any project.
  candidates: loci (another), tensor-serve (using), Delroy (another, using)
```

The abstention was correct, and that is what made it interesting. Every content-bearing part of that question names a *relation between scopes*, and the evidence model is `token -> {scope: node_df}`, so there was nothing to look up. The fact the question wants is not in either store. It is in the registry, and reaching it took three pieces.

Signboards. What a project is called from the outside: repository, distribution, import, command, read from git and root manifests at registration. Section 2.3 is why this is a list rather than a name.

Edges. What a project reaches for, collected two ways, because a real corpus needs both:

```
3m1ry33t-github-io -> urthreads    "urthreads": "^1.2.0"    package.json:10
delroy              -> loci        shutil.which("loci")     client/loci_memory.py:36
```

There is no `.gitmodules` anywhere in the 15-project corpus, no path dependency and no git-URL dependency, so the declared half rests entirely on the plain registry name. And `Delroy -> loci` is a binary resolved on `PATH`, declared in no manifest, lockfile or submodule. **Either collector alone has a recall of 0.5**, which is the argument for building both and is a good example of a design decision that a smaller corpus would have hidden.

A relational path, ahead of routing. Without it the capability is unreachable: `route` can only choose scopes, and this answer is not a scope. Section 14.8 covers why that is a general lesson rather than a special case.

Measured on the current corpus:

```
2 edge(s) from 190 outbound reference(s), collected from 10 of 15 project(s)
nothing to collect from: 3M1RY33T, MyBlog, WEPbridge, brewery, hlep_davay
  - no manifest and no spawn; cannot be the source of an edge
cannot be named by an edge: beacon, hlep_davay
  - no remote, distribution or command
```

Two edges from 190 references is a low yield and the output says so rather than presenting two results as a finding. Naming the five projects that cannot be a source, and the two that cannot be a target, is the same discipline as `doctor`: an empty answer with a reason is worth more than a confident answer from the wrong place.

11. Redaction

Everything collected passes through `loci.redact` **before** it is written, so a credential committed by accident never reaches the episode store, the vectors or the fitted rankers.

The module's own docstring states the reasoning, and it is the right reasoning: the episode store indexes commit bodies, docstrings and comment blocks, three places where secrets get committed by accident and then live forever in history even after the file is fixed. A scan of one real 16,015-chunk store came back clean, **but that is a property of those repositories, not of this tool.**

Files that exist to hold secrets are never opened: `.env*`, `.dev.vars`, `*.pem`, `*.key`, `id_rsa`, `secrets.*`, `credentials.json`. Then every chunk is scanned for AWS keys, GitHub, Slack, Google, Stripe, OpenAI and Anthropic tokens, private-key blocks, JWTs, `user:password@host` connection strings, bearer tokens, and long opaque values assigned to a secret-shaped name.

Three properties make this structural rather than conventional:

Chunks are built through a single constructor, so a new collection path cannot forget to redact. This is the same pattern as Delroy's lane-ownership vocabulary module: the guard lives where it cannot be bypassed by a caller that did not know about it.

A secret assignment keeps its key name and loses only its value, so the chunk still reads sensibly and still routes:

```
client_secret = "aVeryLong..." -> client_secret = "[REDACTED:secret-assignment]"
```

`loci index` reports what it removed, per scope and by kind. Silent redaction is indistinguishable from no redaction.

The bias is deliberate and stated: a false positive costs one chunk a little retrieval quality, a false negative copies a live credential into a plaintext file and a vector index. Those are not comparable costs, so the threshold is not set in the middle.

12. Performance

loci index	15s full · 3.9s when nothing changed
loci ask	1.0s one-shot with <code>--fast</code> · ~5s with semantic ranking
MCP server	6s at boot, then 0.2-0.3s per tool call
routing	sub-millisecond, and flat from 25 to 100 scopes

Three things make that work, and each was a measured bottleneck first.

Traversal prunes, it does not filter. `Path.glob("**/*.py")` descends into `node_modules`, `.venv` and `Pods` in full: 32.9 seconds to enumerate one repository. Pruning improves the index as well as the clock. One project was 66% vendored third-party source before pruning, which means two thirds of its indexed vocabulary was somebody else's.

Lexical rankers are fitted at index time. Character 3-5 gram TF-IDF over a large scope costs about 1.5 seconds, and an in-process cache never survives a CLI or MCP invocation. Fitting at query time would have made every question pay it.

The embedding model is loaded at MCP boot. About 2.3 seconds of imports plus model construction is process startup, not work. `--fast` skips it for one-shot CLI questions, which is why the same query has two very different published timings.

Routing being flat from 25 to 100 scopes is the property that matters for the thesis. The scoring function touches every scope in the corpus on every question, so it *could* have scaled badly, and the dictionary-of-postings representation is what keeps it from doing so.

13. From inside an AI client

`loci skill install` writes one skill with subcommands rather than eight commands cluttering a listing. The passthrough rule is what keeps it short: `index`, `embed`, `graphs`, `calibrate`, `eval` and `groups` all work from the client without being documented twice, so the skill cannot drift from flags it never mentions. What it *does* mention is checked, by a test that parses the Usage block and fails if any command in it stops existing.

The MCP server exposes **three tools**: `ask`, `scopes`, `doctor`. Not several dozen, and the reason is recorded: agents are the primary consumer, and a wide tool surface pushes orchestration onto the model and burns a turn per hop.

Two things the skill teaches an agent that are not obvious from the CLI:

Run it from the project root. `cwd` is the primary routing signal, so a question asked from the wrong directory routes worse than one asked from none.

`ABSTAINED` is an answer. The reason names what to do next, the shortlist names the projects that could plausibly own the question and the terms that put them there, and an agent that rephrases and retries instead is burning turns on a decision that was about vocabulary rather than wording.

14. Design decisions and their reasons

Each of these is something a reasonable implementation gets wrong.

14.1 Never merge scopes into one index. Section 3. 2% and 18% on-topic against a merged graph, and 98% on the question where the biggest project happened to be right.

14.2 Abstention is a feature at both layers. Measured 10 of 10 on probes including deliberate nonsense. Every retrieval tool confidently answers "what is the airspeed velocity of an unladen swallow". This one says it does not know.

14.3 Margin and ratio tests do not measure confidence. Both tried, both inverted on real data. Section 8.2.

14.4 A matched-token count is not evidence. It does not separate routable from unroutable and can invert. Summed evidence does.

14.5 Deixis is grammatical, so detect it grammatically. A question that points at its subject cannot be routed by vocabulary, because the words that would identify the subject are exactly the ones it declines to say. No lexical statistic rescues this: per-token evidence for `start` (2.51) and `services` (2.56) is indistinguishable from real evidence like `session` (2.53). A closed class of markers took abstention from 37.5% to 100%.

14.6 Normalize for scope size, twice. With one scope 67 times larger, ordinary English words appear in exactly one scope and score as maximally discriminative for it.

14.7 Docstrings are excellent retrieval material and poor routing material. They are high-volume generic English, "returns", "the value", "default", "configuration", and folding them into routing halved accuracy on both a symbol-indexed and a prose-only corpus. The same text is kept for retrieval and excluded from routing, which is a distinction most systems do not have the structure to make.

14.8 Coverage is the binding constraint, not ranking. This surfaced in every experiment. Running a free AST-only index over three unindexed repositories moved routing from 69.6% to 82.6% *while adding two more scopes to compete against*. Hence `doctor` as a first-class command: an empty answer with a reason beats a confident answer from the wrong project.

14.9 Nothing updates itself. No watcher, no daemon, no installed git hook. Every refresh is a command. The one thing `loci update` rebuilds unconditionally is the structure graph, because that is the only staleness in loci that is otherwise silent: `loci graphs` builds a graph that is *missing* and skips every scope that already has one, so a project whose code moved would keep routing on the symbols it had the day it was registered. Everything else announces itself.

15. Open problems

15.1 Aisles inside a shop. A word found in a test fixture counts like the same word in shipped code. That is wrong in a way that is easy to state and hard to fix without a second layer of scoping inside each scope, and it is the largest known source of vocabulary noise.

15.2 The `contended` family at 75%. Section 9.2. The family that tests the case the whole tool exists for is the weakest one measured, and three of twelve questions where two scopes should both come back returned only one. The set-selection rule of Section 5.2 was written for this case; it is evidently not sufficient for it.

15.3 The constants were fitted on ten scopes and now run against fifteen. `signature` fell from 100% to 93.3% over that growth. Nothing about that is alarming on its own, but the shipped evidence floor of 7.6 against a fitted 5.411 on this corpus (Section 9.3) says the default is now noticeably off for the corpus it was developed on, which is the corpus most likely to resemble a first user's.

15.4 A deleted project stays in the registry. Reported, not removed, because the entry carries groups and aliases set by hand and the directory may only be unmounted. That is the right default and it is not a complete answer.

15.5 One test file. 5,446 lines of tests in a single `tests/test_core.py`. The coverage is real and the suite is green, but the organisation gives no signal about which subsystem a failure belongs to, and it will not survive another few thousand lines.

16. Conclusion

loci is a small tool with one idea, and the idea survived contact with measurement.

The idea is that scope is a read-time decision. Everything else follows: the router exists because the decision has to be made somewhere, the abstention exists because the decision is sometimes not makeable, the shortlist exists because an unmakeable decision should be handed back with its working shown, and `doctor` and `eval` exist because a decision fitted to one person's corpus should be checkable against anyone else's.

What is worth taking from it as engineering, rather than as a tool, is the pattern of the findings in Section 14. Six of the nine record something that was implemented, measured, and removed. A margin test, a ratio test, a matched-token count, docstrings in the routing index: each was the obvious thing to try, each was tried, and each is now a paragraph explaining why it is not in the code. **The alternative is a codebase where the obvious wrong answer is absent and nobody remembers whether it was rejected or never considered**, and the difference between those two states is the whole of whether the next person can build on the work.

Appendix A: Verification log

Measured against the working tree at commit `a8a4673` on 8 September 2026.

A.1 Tests

```
$ python3 -m pytest tests -q
279 passed in 26.50s
EXIT=0
```

A.2 Scale

```
$ git ls-files '*.py' | grep -v '^tests/' | xargs wc -l | tail -1
8105 total

$ git ls-files '*.py' | grep -v '^tests/' | wc -l
29

$ git ls-files 'tests/*.py' | xargs wc -l | tail -1
5446 tests/test_core.py

$ git rev-list --count HEAD
85

$ git log --reverse --format='%ad' --date=short | head -1
2026-08-27

$ git log -1 --format='%h %ad %s' --date=short
a8a4673 2026-09-06 release: 0.5.0 -- a question about the road between two projects
```

A.3 Routing evaluation

```
$ loci eval
15 scopes | random guessing would score 6.7%

family          n correct scopes what it measures
deictic + cwd   60 100.0%    - should route to the scope you are in
deictic, no cwd  4 100.0%    - should abstain; nothing says which project
unanswerable    3 100.0%    - should abstain; no corpus can answer these
signature       15 93.3%     1.3 are your scopes distinguishable from each other?
detailed        30 86.7%     - longer questions, ordinary vocabulary
contended       12 75.0%     1.7 two scopes share the term; both should come back

routing looks healthy on this corpus.
```

A.4 Calibration

```
$ loci calibrate --show
evidence floor      5.411
semantic floor      0.639 (mean per-scope, 8 scopes fitted, bands separate by +0.114)
fitted from         155 routable + 7 unroutable questions, none hand-labelled
routable band       from 2.507
unroutable band     up to 5.092
classifies          93.8% of its own samples correctly
```

A.5 Cross-project edges

```
$ loci uses
3M1RY33T.github.io -> urthreads (depends on `urthreads`) package.json:10
Delroy -> loci (runs `loci`) client/loci_memory.py:36

2 edge(s) from 190 outbound reference(s), collected from 10 of 15 project(s)
```

A.6 Routing constants

From `src/loci/router.py`, lines 26 to 35:

```
ALIAS_BOOST = 6.0      # the question names the project outright
CWD_BOOST = 4.0        # cwd is inside the project tree
REGENCY_BOOST = 0.15   # tiebreak only; must never outrank real evidence
GROUP_PENALTY = 0.5    # multiplicative on the evidence base
WIDEN_RATIO = 0.85     # keep scopes scoring >= WIDEN_RATIO * top
MAX_SCOPES = 3
MIN_MATCHED = 4        # legacy count gate; see EVIDENCE_FLOOR
EVIDENCE_FLOOR = 7.6   # summed token evidence required to route at all
CONCENTRATED_SCOPES = 2 # a token held by at most this many scopes
CONCENTRATED_EVIDENCE = 2.6
```

Appendix B: What it writes

Everything except one file goes to `~/.loci` (or `$LOCI_HOME`). One directory, not one per project: routing is a cross-scope decision, so answering *which project is this about* needs every scope's vocabulary in the same lookup, and N per-project indexes would turn the cheap question into the expensive one.

```
~/.loci/
├─ scopes.json           the registry, and the roots it was scanned from
├─ groups.json           group policy: default mode, and any per-group mode
├─ scope_index.json      the routing index, token -> {scope: node_df}
├─ episodes.json         the episode store, chunks verbatim, already redacted
├─ rankers/<scope>.joblib fitted BM25 + char-gram
├─ embeddings.npz        chunk and symbol vectors
├─ .symbols-<scope>.json symbol labels, in the order embeddings.npz has
├─ calibration.json      the fitted evidence floor, and a semantic floor
├─ .index.lock           advisory build locks, so two builds cannot
└─ .embed.lock           interleave their writes
```

Inside a repository exactly two files matter, and they point in opposite directions:

```
<repo>/graphify-out/graph.json    loci writes it, on request. You never do.
<repo>/~.loci.json                you write it. loci never does.
```

A malformed `.loci.json` is ignored rather than fatal. A scan that aborts on a stray comma is worse than one that misses a declaration.

Prepared 8 September 2026 against the working tree at commit `a8a4673`, release 0.5.0. Scale, test and git figures were measured during preparation and Appendix A reproduces the commands. The routing, calibration and edge figures are live output from the 15-scope development corpus, run during preparation. Historical measurements quoted in Sections 3, 5, 6, 7, 8, 12 and 14 are the project's own recorded findings from its README and changelog, taken from the runs that produced the constants, and are attributed as such rather than re-derived.